

课程须知

- 本课程适用软件版本：MWORKS.Syslab2024a
- 本课程示例运行需要软件首选项加载：基础库、数学库、图形库、通信库、控制系统库、信号处理库

MWORKS.Syslab通信工具箱应用

周志恒

苏州同元软控信息技术有限公司

2025年5月23日



TONGYUAN
Software and Control

目录

1. 通信工具箱功能概况

2. 信源编码

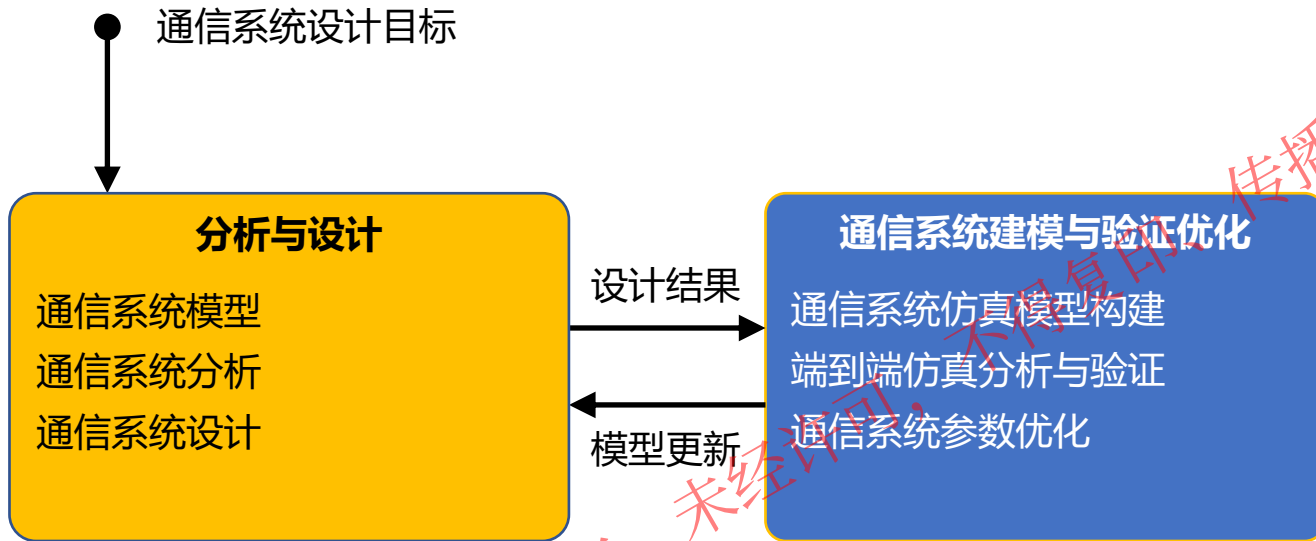
3. 信道编码

4. 频带信号的发送与接收

5. 性能仿真分析

6. 通信系统仿真综合案例

1. 通信工具箱功能概述

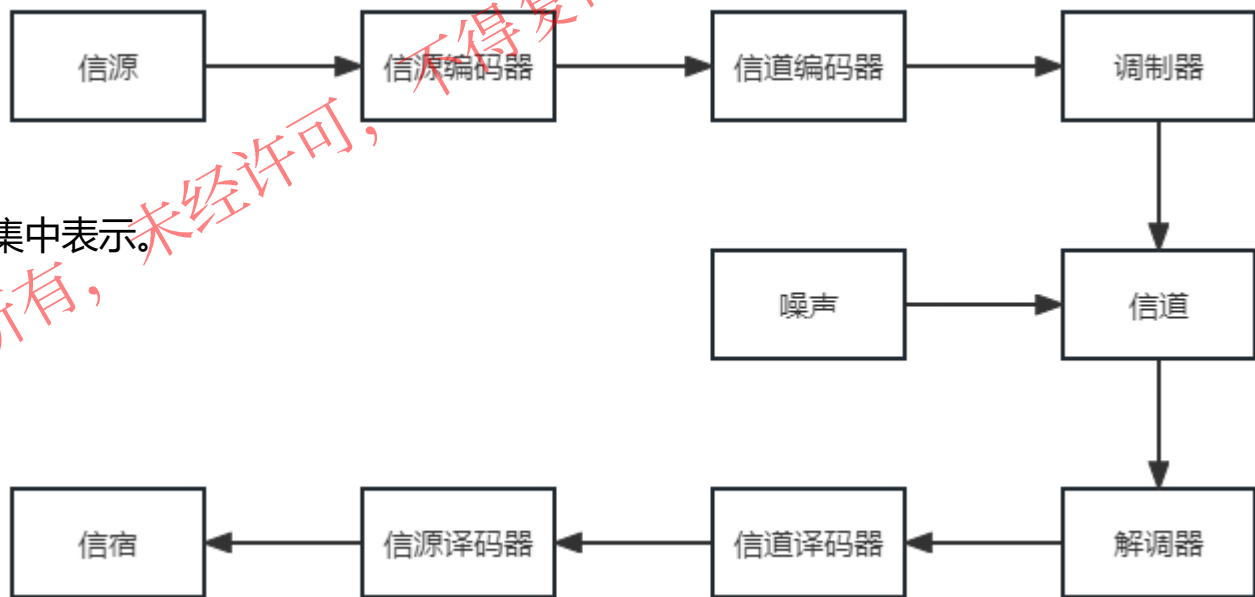


MWORKS控通信工具箱旨在为通信系统**分析与设计**阶段提供计算支撑

1.1 数字通信系统模型

数字通信系统模型的组成、各部分的功能。

- ◆ **信源**是发送消息的源，有数字信源和模拟信源之分。
- ◆ **信宿**是收信者或用户，是信息传送的终点或目的地。
- ◆ **信道**是信息的传输通道。
- ◆ **噪声**是信道中的噪声及分散在通信系统其他各处的噪声的集中表示。
- ◆ **信源编码**是为了去掉信源中的信息冗余。
- ◆ **信源译码**是信源编码的逆过程。
- ◆ **信道编码**的主要作用是提高信息传送的可靠性。
- ◆ **信道译码**是信道编码的逆过程。



1. 通信系统工具箱功能概述

1 物理层组件建模

- 物理层组件建模
- 表征通信链路

2 射频元件建模

- 射频无线电建模
- 损伤校正

3 传播和信道模型

- 基于规范传播模型
- 衰落信道模型

4 测试和测量

- 波形生成
- 可视化
- 性能分析

目录

1. 通信工具箱功能概况

2. 信源编码

3. 信道编码

4. 频带信号的发送与接收

5. 性能仿真分析

6. 通信系统仿真综合案例

2 通信系统- 信源编码

多数信源(比如语音、图像)都是模拟信号, 为了将信源输出数字化, 信源必须量化为确定数目的级数。

- 标量量化: 每个信源输出都分别被量化
 - 均匀量化: 量化区域是等长的
 - 非均匀量化: 量化区域可以是不等长的
- 矢量量化: 信源输出组合进行整体量化

一. 标量量化

标量量化中, 随机标量 X 的定义域被划分成 N 个互不重叠的区域 R_i ; $1 \leq i \leq N$, R_i 被称为量化间隔, 且在每个区域内选择一个点作量化级数。区域 R 内的随机变量的所有值都被量化为第 i 个量化级数, 用 x_i 来表示:

$$x \in R_i \Rightarrow Q(x) = \hat{x}_i$$

量化引入了失真, 均方误差为:

$$D = \sum_{i=1}^N \int_{R_i} (x - \bar{x}_i)^2 f_x(x) dx$$

其中 $f(x)$ 是信源随机变量的概率密度函数。信号量化噪声比(SQNR)为:

$$SQNR = 10 \log_{10} \frac{E(X^2)}{D}$$

2 通信系统- 信源编码

一. 标量量化

示例1: Syslab中对信号压缩扩张:

实现 μ 律压缩扩张

$$y = \frac{\log(1 + \mu|x|)}{\log(1 + \mu)} \operatorname{sgn}(x)$$

$$y^{-1} = \operatorname{sgn}(y) \frac{1}{\mu} ((1 + \mu)^{|y|} - 1)$$

```
# 生成数据序列
data = collect(2:2:12)
# 使用  $\mu$  - 律压缩器压缩数据序列,将  $\mu$  的值设置为 255.
compressed = compand(data, 255, maximum(data), "mu/compressor")
# 使用  $\mu$  - 律展开器展开压缩数据序列
expanded = compand(compressed, 255, maximum(data), "mu/expander")
# 计算原始数据序列与扩展序列之间的差值
diffvalue = expanded - data
```

运行结果

```
6-element Vector{Float64}:
-4.440892098500626e-16
 1.7763568394002505e-15
 8.881784197001252e-16
 3.552713678800501e-15
 1.7763568394002505e-15
-3.552713678800501e-15
```

A律压缩扩张

$$y = \begin{cases} \frac{A|x|}{1 + \log(A)} \operatorname{sgn}(x) & 0 \leq |x| \leq \frac{1}{A} \\ \frac{(1 + \log(A|x|))}{1 + \log A} \operatorname{sgn}(x) & \frac{1}{A} < |x| < 1 \end{cases}$$

$$y^{-1} = \operatorname{sgn}(y) \begin{cases} \frac{|y|(1 + \log(A))}{A} & 0 \leq |y| \leq \frac{1}{1 + \log(A)} \\ \frac{\exp(|y|((1 + \log(A)) - 1))}{1 + \log A} & \frac{1}{1 + \log(A)} < |y| < 1 \end{cases}$$

```
# 生成数据序列
data = collect(1:1:5)
# 使用 A - 律压缩器压缩数据序列,将 A 的值设置为 87.6
compressed = compand(data, 87.6, maximum(data), "A/compressor")
# 使用 A - 律展开器展开压缩数据序列
expanded = compand(compressed, 87.6, maximum(data), "A/expander")
# 计算原始数据序列与扩展序列之间的差值
diffvalue = expanded - data
```

运行结果

```
5-element Vector{Float64}:
 0.0
 0.0
 1.3322676295501878e-15
 8.881784197001252e-16
 8.881784197001252e-16
```

2 通信系统- 信源编码

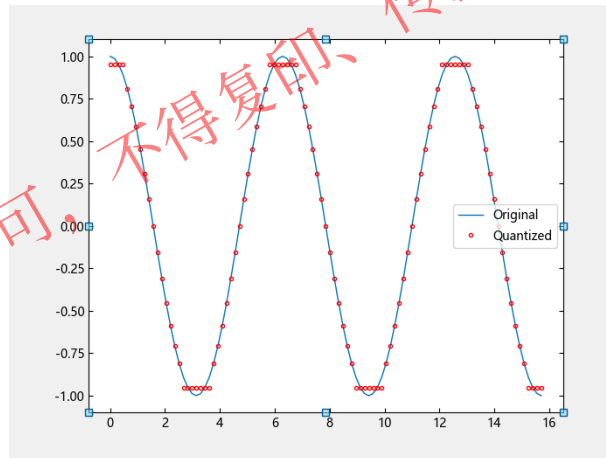
一. 标量量化

示例2: Syslab中用lloyds优化标量算法

对一个余弦数据进行标量化

```
ini_codebook = 2^4
# 生成正弦信号
t = (0:100) * pi ./ 20
training_set = cos.(t)
# 生成分界点partition和codebook
partition, codebook = lloyds(training_set, ini_codebook)
# 量化信号
index, quant, distort = quantiz(training_set, partition, codebook)
# 绘制量化信号及原信号
plot(t, training_set, t, quant, "r.")
legend("original", "Quantized")
```

运行结果:



二. 信源编码-DPCM

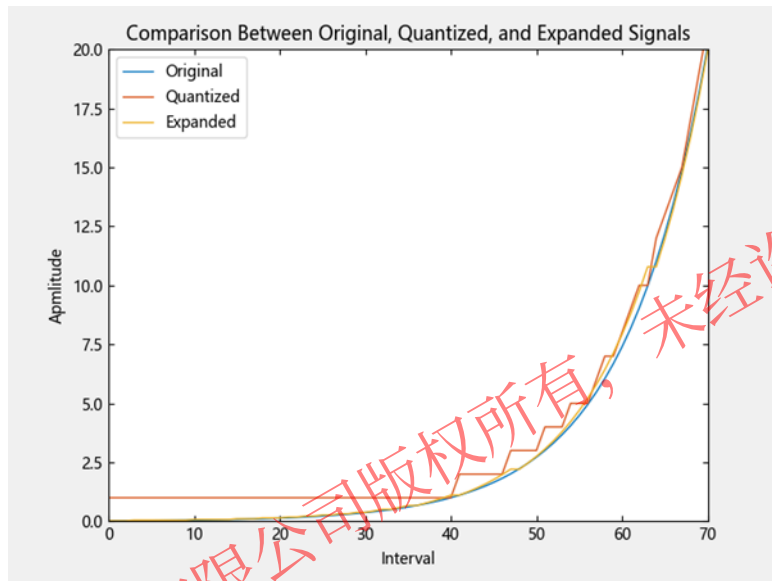
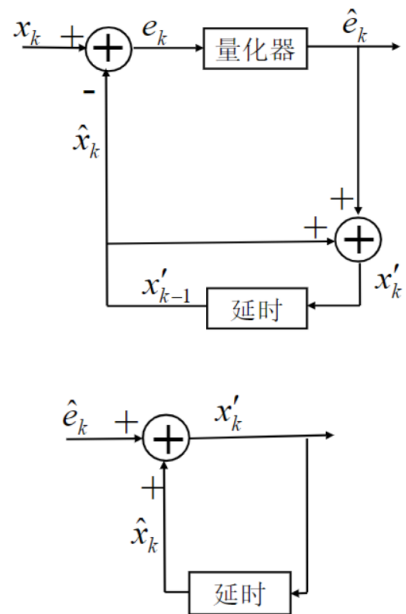
信源编码是一种以提高通信有效性为目的而对信源符号进行的变换,或者说为了减少或消除信源冗余度而进行的信源符号变换。

差分脉冲编码调制 (DPCM) 是信源编码的一种,用于模拟信号的编码模式,与PCM不同每个抽样值不是独立的编码,而是先根据前一个抽样值计算出一个预测值,再取当前抽样值和预测值之差作编码用。抽样值和预测值非常接近(因为相关性强),预测误差的可能取值范围比抽样值变化范围小,所以可用少几位编码比特来对预测误差编码,从而降低其比特率。这是利用减小冗余度的办法,降低了编码比特率。

2 通信系统- 信源编码

二. 信源编码-DPCM

差分脉冲调制编码器与解码器



示例3: Syslab中对指数信号压缩、量化、扩展

```
# 生成指数信号
sig = exp.(-4:0.1:4)
V = maximum(sig)
# 量化指数信号
partition = 0:2^6-1
codebook = 0:2^6
_, qsig, distortion = quantiz(sig, partition, codebook)
# 压缩-量化-扩展指数信号
mu = 255 # mu-law parameter
csig_compressed = compand(sig, mu, V, "mu/compressor")
_, quants = quantiz(csig_compressed, partition, codebook)
csig_expanded = compand(quants, mu, maximum(quants), "mu/expander")
distortion2 = sum((csig_expanded - sig).^2) ./ length(sig)
# 比较均方误差
[distortion distortion2]
# 绘图
hold("on")
plot([sig, qsig, csig_expanded])
title("Comparison Between Original, Quantized, and Expanded Signals")
xlabel("Interval")
ylabel("Amplitude")
legend(["Original", "Quantized", "Expanded", "location", "nw"])
axis([0 70 0 20])
```

运行结果

```
julia> [distortion distortion2]
1x2 Matrix{Float64}:
 0.534785  0.0396674
```

- (1)输入信号为 x_k , $\hat{x}_k$ 为根据前 $k-1$ 个信号预测的第 k 个信号, e_k 为输入值与预测值之间的误差
- (2) e_k 经过一个量化器, 量化为 $\hat{e}_k$
- (3)编码 $\hat{e}_k$ 并作为输出
- (4)量化误差 $\hat{e}_k$ 和当前输入信号的预测值 $\hat{x}_k$ 求和, 得到重建信号 x'_k
- (5)当前重建信号与前 k 个重建信号线性组合, 得到下一个信号预测值 $\hat{x}_{k+1}$
- (6) $\hat{x}_{k+1}$ 与 x_{k+1} 进行下一轮编码, 重复上述流程

2.1 通信系统- 信源编码

信源编码相关主要Syslab函数，其余函数请参阅Syslab-通信工具箱帮助文档

函数及调用方式	说明
arithdeco	通过算术解码来解码二进制代码
arithenco	通过算术编码对符号序列进行编码
compand	信源编码中的 μ - 律或 A - 律压缩扩展器
dpcmdeco	使用差分脉冲编码调制进行解码
dpcmenco	使用差分脉冲编码调制进行编码
dpcmopt	优化差分脉冲编码调制参数
lloyds	使用劳埃德算法优化量化参数
quantiz	产生量化指标和量化输出值

目录

1. 通信工具箱功能概况

2. 信源编码

3. 信道编码

4. 频带信号的发送与接收

5. 性能仿真分析

6. 通信系统仿真综合案例

3 通信系统- 信道编码

差错控制编/译码技术被用于检查和纠正信息在传递过程中发生的错误。在发送端，差错控制编码添加了一定的冗余码元到信源序列;接收时利用这些冗余信息来检测和纠正错误。常用的纠错编码方法包括线性分组码、海明码、循环码、BCH码、Reed-Solomon码和卷积码等。

一. 线性分组码

分组码是把信息码元序列以每 k 个码元分组，编码器将每个信息组按照一定规律产生 r 个多余的码元（称为校验元），形成一个长为 $n=k+r$ 的码字 (n, k) 。**线性分组码**的信息码元与监督码元之间的关系为线性关系。

汉明码是线性分组码的一种，可纠正一个错码或者检测两个错码，以 $(7, 3)$ 汉明码为例：

汉明码一个码字中有7个码元，监督码元与信息码元之间的关系：

$$\begin{cases} a_6 \oplus a_5 \oplus a_4 \oplus a_2 = 0 \\ a_6 \oplus a_5 \oplus a_3 \oplus a_1 = 0 \\ a_6 \oplus a_4 \oplus a_3 \oplus a_0 = 0 \end{cases}$$

$a_6 - a_3$ 为信息码元, $a_2 - a_0$ 为监督码元。

示例1: Syslab中生成校验矩阵和生成矩阵

```
# 为码字长度为 7 的汉明码生成校验矩阵H和生成矩阵G
H,G = hamngen(3)
```

运行输出：

```
julia> H
3x7 Matrix{Int64}:
 1  0  0  1  0  1  1
 0  1  0  1  1  1  0
 0  0  1  0  1  1  1
```

```
julia> G
4x7 Matrix{Int64}:
 1  1  0  1  0  0  0
 0  1  1  0  1  0  0
 1  1  1  0  0  1  0
 1  0  1  0  0  0  1
```

3 通信系统-信道编码

一. 线性分组码

示例2： Syslab中使用**汉明解码**表来纠正一个收到的信息中的错误

hammgen 函数产生奇偶校验矩阵，syndtable 函数产生解码表。为了确定校验子，奇偶校验矩阵的转置在左边乘以收到的码字。解码表有助于确定校正向量。修正后的码字是修正向量和接收码字的总和（模数为2）。

运行结果：

```
# 为 [7,4] 汉明码设置参数
m = 3
n = 2^m - 1
k = n - m
# 产生一个奇偶校验矩阵和解码表
parmat, = hammgen(m)
trt = syndtable(parmat)
# 指定一个接收数据的向量
recd = [1 0 0 1 1 1 1]
# 计算校验子，然后显示校验子的十进制和二进制值
syndrome = rem.(recd * parmat', 2)
syndrome_int = bi2de(syndrome, "left-msb")
println("Syndrome = " * string(syndrome_int) * " decimal, " * string(syndrome) * " binary")
# 通过使用解码表和校验子确定校正向量
correct = trt[1+syndrome_int, :]
# 然后通过校正向量计算出校正后的码字
correctedcode = rem.(correct .+ vec(recd), 2)
```

```
julia> [recd' correctedcode]
7x2 Matrix{Int64}:
 1  1
 0  0
 0  0
 1  1
 1  0
 1  1
 1  1
```


3 通信系统-信道编码

一. 线性分组码

BCH码是循环码的子类，是一类有限域中的纠错码，它把信源待发的信息序列按固定的k位一组划分成消息组，再将每一消息组独立变换成长度为n(n>k)的码字，具有纠正多个随机错误的能力。BCH码纠错能力很强，在短和中等码长下，其性能接近于理论值，并且构造方便，编码简单。

对于一个分组长度 $n = p^m - 1$ ，确定可纠正t个错误的BCH码的生成多项式的步骤如下：

(1)选取一个次数为m的素多项式并构造 $GF(p^m)$

(2)求 $a^i, i = 0, 1, 2, \dots, n-2$ 的极小多项式 $f_i(x)$

(3)可纠正t个错误的码的生成多项式：

$$g(x) = LCM[f_1(x), f_2(x), f_3(x) \dots f_{2t}(x)]$$

$d = 2t + 1$ 称为码的设计距离，一旦确定了n和t，BCH码的生成多项式就确定了

示例3: Syslab中创建BCH生成多项式

```
# 设置码字和消息长度 n 和 k
N = 15
K = 11
# 创建生成多项式并返回纠错能力 t
genpoly1, t = bchgenpoly(15, 11)
# 使用不同本原多项式为 (15,11) BCH 码创建生成多项式
genpoly2, = bchgenpoly(15, 11, 19)
```

运行结果:

```
julia> genpoly2
5-element Vector{GF11}:
 1
 0
 0
 1
 1
```

示例4: 计算BCH码的可纠正错误

```
# 计算 BCH 码字长度为 15 的可能消息长度组合
T1 = bchnumerr(15)
# 计算 BCH 码 15,11 的可纠正错误数
T2 = bchnumerr(15, 11)
```

运行结果:

```
julia> T1
3x3 Matrix{Int64}:
 15 11 1
 15 7 2
 15 5 3

julia> T2
1
```

示例5: Syslab中在噪声信道中解码接收的 BCH 码字

```
# 为 GF(2) 的 Galois 数组设置 BCH 参数
rng = MT19937ar(1234)
M = 4
n = 2^M - 1
k = 5
nwords = 10
# 创建消息
msgTx = GF1.(randi(rng, [0 1], nwords, k))
# 查找纠错能力
t = bchnumerr(n, k)
# 对消息进行编码
enc = bchenc(msgTx, n, k)
# 每个码字中最多损坏 t 比特
noisycode = enc + GF1.(randerr(nwords, n, collect((1:t))))
# 解码有噪声的码字
msgRx, = bchdec(noisycode, n, k)
# 验证消息是否已正确解码
println(isequal(msgTx, msgRx))
```

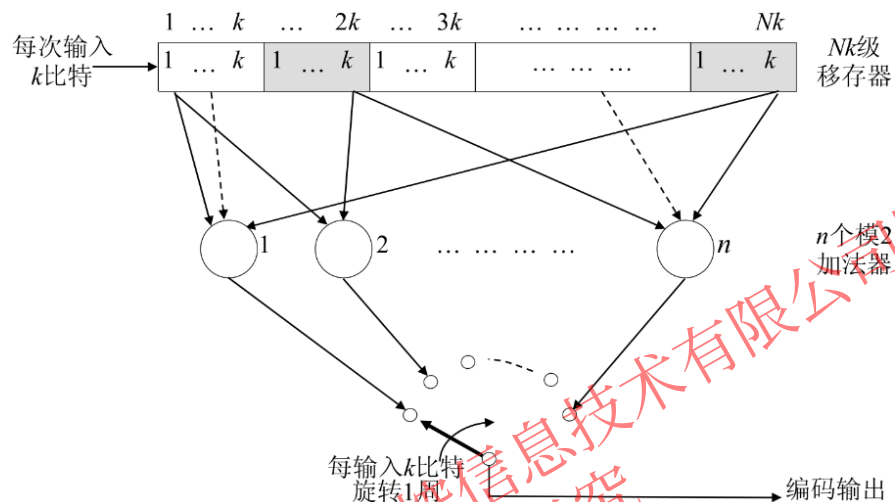
运行结果:

```
julia> println(isequal(msgTx, msgRx))
true
```

3 通信系统-信道编码

二. 卷积码

卷积码是一种非分组码。通常适用于前向纠错。卷积码在编码时把 k 个比特的信息段编成 n 个比特的码组，监督码元不仅和当前的 k 比特信息段有关还同前面 $m=(N-1)$ 个信息段有关。一个码组中的监督码元监督着 N 个信息段，通常将 N 称为编码约束度并将 nN 称为编码约束长度。卷积码记 (n,k,N) ，码率则定义为 k/n 。



示例6:在 Syslab 中创建卷积码

```
rng = MT19937ar(1234)
# 使用 poly2trellis 函数定义用于配置编码器的 trellis 结构
trellis_a = poly2trellis([5 4], [23 35 0; 0 5 13])
# 使用 trellis_a 结构来配置 convenc 函数
K = log2(trellis_a.numInputSymbols)
N = log2(trellis_a.numOutputSymbols)
numReg = log2(trellis_a.numStates)
numSymPerFrame = 5
K = convert(Int64, K)
numSymPerFrame = convert(Int64, numSymPerFrame)
data = rand(rng, 0:1, K * numSymPerFrame)
# 以 2/3 的码率为五个二进制符号进行编码
code_a, fstate_a = convenc(data, trellis_a)
# 验证编码输出是否为 15 位，即输入序列数据长度的 3/2 (N/K) 倍
k = length(data) // length(code_a)
```

运行结果:

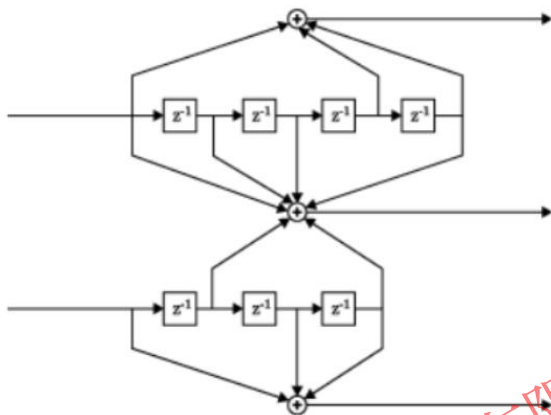
```
julia> code_a
15x1 Matrix{Int64}:
 0
 0
 1
 0
 ⋮
 1
 0
 0

julia> k
2//3
```

3 通信系统-信道编码

二. 卷积码

维特比解码是一种概率解码。将接收到的信号序列和所有可能的发送信号序列比较，选择其中汉明距离最小的序列认为是当前发送信号序列。其算法所保留的路径与接收序列之间的似然概率为最大，所以又称为最大似然译码。



示例7: Syslab中2/3 速率卷积码误码率估计

```
Random.seed!(1234)
# 定义图中表示的卷积编码网络
trellis1 = poly2trellis([5 4], [23 35 0; 0 5 13])
K = log2(trellis1.numInputSymbols)
N = log2(trellis1.numOutputSymbols)
coderate = K / N
fprintf("K is %d and N is %d. The code rate is %3.2f\n", K, N, coderate)
```

```
# 设置调制顺序，并计算每个调制符号的比特数
M = 16
bps = log2(M)
numSymPerFrame = 5000
dataIn = randi([0 1], trunc(Int64, K * bps * numSymPerFrame), 1)
# 对输入数据进行卷积编码
codedout, = convenc(dataIn, trellis1)
# 对编码符号应用 16-QAM 调制
txSig = qammod(codedout, M, InputType="bit")
# 计算 awgn 函数使用的信噪比值
EbNo = 9
snr1 = EbNo + 10 * log10(bps * coderate)
# 通过AWGN信道
rxSig = awgn(txSig, snr1, "measured")
# 解调接收到的信号
demodSig = qamdemod(rxSig, M, OutputType="bit")
# 指定维特比解码器的回溯深度
tbdepth = 16
# 使用维特比解码器解码二进制解调信号
dataOut, = vitdec(demodSig, trellis1, tbdepth, "cont", "hard")
# 将编码的误码率与理论的未编码的误码率进行比较
decDelay = K * tbdepth
decDelay = trunc(Int64, decDelay)
berCoded = biterr(dataIn[1:end-decDelay], dataOut[decDelay+1:end]) ./ length(dataOut[decDelay+1:end])
length(dataOut[decDelay+1:end])
berUncoded = berawgn(EbNo, "qam", M)
fprintf("The coded BER is %6.5f\nThe uncoded BER is %6.5f\n", [berCoded[1] berUncoded[1]])
```

运行结果

K is 2 and N is 3. The code rate is 0.67
The coded BER is 0.00048
The uncoded BER is 0.00439

与未编码的误码率相比，卷积码编码解码后的误码率低一个数量级，能起到较好的差错控制的作用。



3 通信系统-信道编码

信道编码相关主要Syslab函数，其余函数请参阅Syslab-通信工具箱帮助文档

函数及调用方式	说明
cyclgen	为循环码创造奇偶校验和生成矩阵
cyclpoly	为循环码创造生成多项式
dvbs2ldpc	DVB-S.2标准的低密度奇偶校验（LDPC）码
bchdec	BCH 解码器
bchenc	BCH 编码器
convenc	卷积编码二进制信息
poly2trellis	将卷积码多项式转换为 trellis 描述形式
vitdec	使用维特比算法对二进制数据进行卷积解码



目录

1. 通信工具箱功能概况

2. 信源编码

3. 差错控制编码

4. 频带信号的发送与接收

5. 性能仿真分析

6. 通信系统仿真综合案例

4 通信系统-频带信号的发送与接收

基带信号通过调制转换成频带信号。根据调制信号的不同，调制分为**模拟调制**和**数字调制**。模拟调制的输入信号为连续变化的模拟量，数字调制的调制信号是离散的数字量。

一. 模拟调制与解调

模拟调制指利用输入的模拟信号直接调制(或改变)载波(正弦波)的振幅、频率或相位，从而得到调幅(AM)、调频(FM) 或调相(PM) 信号。

➤ 调制信号

模拟信号~ $m(t)$

➤ 载波

连续正弦波~ $s(t) = A \cos(\omega_c t + \varphi_0)$

➤ 已调信号

$$s_m(t) = A(t) \cos[\omega_c t + \varphi(t)]$$

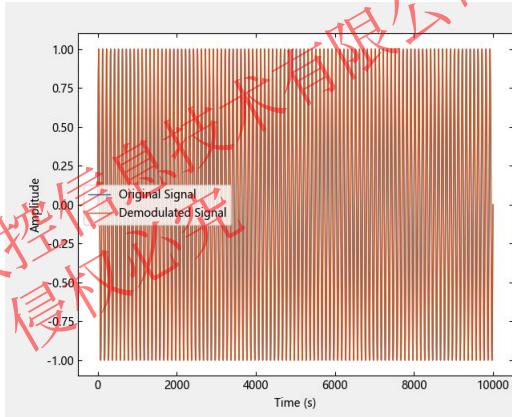
$A(t)$ ~已调信号的瞬时幅度

$\varphi(t)$ ~已调信号的瞬时相位

幅度调制: $A(t)$ 随 $m(t)$ 成比例变化~线性调制

相位调制: $\varphi(t)$ 随 $m(t)$ 成比例变化~角度调制

频率调制: $\frac{d\varphi(t)}{dt}$ 随 $m(t)$ 成比例变化~非线性调制



调幅 (AM): 调幅就是用调制信号去控制高频载波信号的幅度。

示例1: Syslab中调制解调AM信号

```
fs = 100
t = (0:1/fs:100)
fc = 10
# 一个正弦信号
x = sin.(2 * pi * t)
# 使用 ammod 调制正弦信号
ydouble = ammod(x, fc, fs)
# 使用 amdemod 解调
x1 = amdemod(ydouble, fc, fs)
# 绘制原始信号、调制解调后的信号图
figure()
plot(x)
hold("on")
plot(x1)
legend(["Original Signal", "Demodulated Signal"])
xlabel("Time (s)")
ylabel("Amplitude")
```

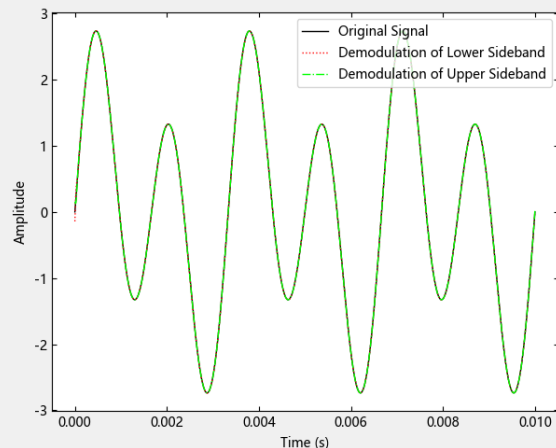

4 通信系统-频带信号的发送与接收

一. 模拟调制与解调

示例2: Syslab中单边带信号的解调

```
# 设置采样频率
fs = 270000;
t = (0:1/fs:0.01)';
# 生成原始信号
signal = sin.(2 * pi * 300 .* t) .+ 2 * sin.(2 * pi * 600 .* t);
# 使用 12000 的截止频率和 0 的初始相位
fc = 12000;
initialPhase = 0;
# 使用 ssbmod 将原始信号转换为上边带和下边带调制信号
lowerSidebandSignal = ssbmod(signal, fc, fs, initialPhase);
upperSidebandSignal = ssbmod(signal, fc, fs, initialPhase, "upper");
# 解调下边带和上边带信号
s1 = ssbdemod(lowerSidebandSignal, fc, fs);
s2 = ssbdemod(upperSidebandSignal, fc, fs);
```

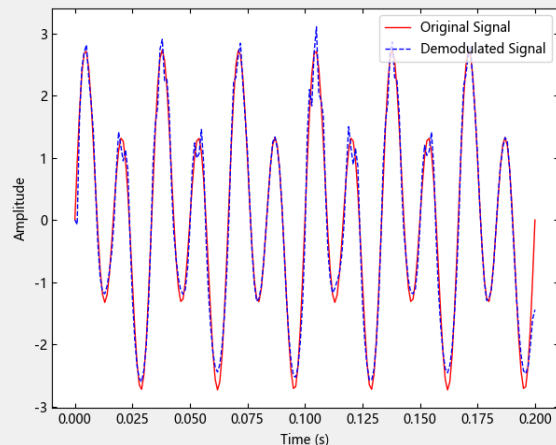
```
# 将处理后的信号与原始信号进行比较并验证重建
figure(1)
plot(t, signal, "k", t, s1, "r:");
hold("on")
plot(t, s2, "-."; color=(0, 1, 0))
hold("off")
xlabel("Time (s)")
ylabel("Amplitude")
legend([
    "Original Signal",
    "Demodulation of Lower Sideband",
    "Demodulation of Upper Sideband"
]);
```



示例3: Syslab中FM调制和解调正弦信号

```
# 将采样频率设置为 1kHz, 将载波频率设置为 200Hz
fs = 1000;
fc = 200;
# 生成一个持续时间为 0.2s 的时间向量
t = 0:1/fs:0.2;
# 生成两个频率为 30 和 60 Hz 的正弦信号并求和
x = sin.(2 * pi * 30 * t) .+ 2 * sin.(2 * pi * 60 * t);
# 将频率偏差设置为 50 Hz
fDev = 50
```

```
# 对信号 x 进行频率调制
y = fmmmod(x, fc, fs, fDev)
# 对信号 y 进行频率解调
z = fmdemod(y, fc, fs, fDev)
# 绘制原始信号和解调信号
plot(t, x, "r", t, z, "b--");
xlabel("Time (s)")
ylabel("Amplitude")
legend(["Original Signal", "Demodulated Signal"])
```



调频 (FM) : 调频就是用调制信号x去控制高频载波信号的频率。

4 通信系统-频带信号的发送与接收

一. 模拟调制与解调

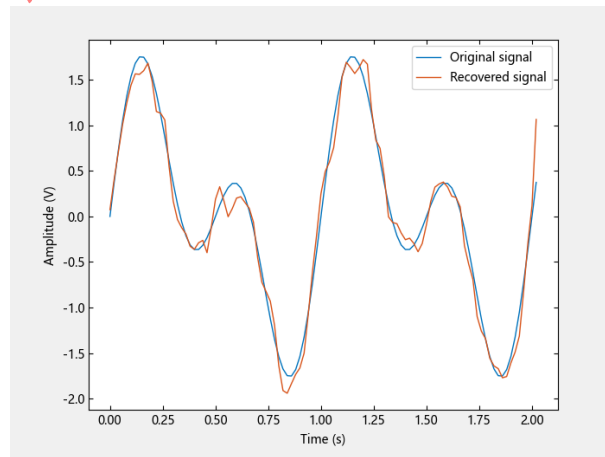
调相 (PM) : 调相就是用调制信号去控制高频载波信号的相位。

示例4: Syslab中从高斯白噪声信道中恢复相位调制信号

```
# 将采样频率设置为 50Hz, 生成时间向量
Random.seed!(123)
fs = 50
t = (0:2*fs+1) / fs
# 生成正弦输入信号
x = sin.(2 * pi * t) + sin.(4 * pi * t)
# 设置载波频率和相位偏移
fc = 10
phasedev = pi / 2
# 对信号x进行相位调制
y = pmmmod(x, fc, fs, phasedev)
```



```
# 将调制后信号加入高斯白噪声
yn = awgn(y, 10, "measured")
# 对信号解调
z = pmdemod(yn, fc, fs, phasedev)
# 绘制原始信号和解调信号
figure()
plot(t, [x z])
legend(["Original signal", "Recovered signal"])
xlabel("Time (s)")
ylabel("Amplitude (V)")
```



二. 数字调制与解调

数字调制即用基带数字信号控制高频载波的振幅、频率和相位中的一种特性或多种, 把基带数字信号变换为频带数字信号的过程。

包括多进制幅度键控 (M-ASK)、多进制频率键控(M-FSK)以及多进制相位键控(M-PSK)等。

4 通信系统-频带信号的发送与接收

二. 数字调制与解调

多进制频移键控信号 (M-FSK) : 正弦载波的频率随数字基带信号而变化的数字调制。

2-FSK: 二进制振幅键控信号, 传0信号时, 发送频率为 f_1 的载波; 传1信号时, 发送频率为 f_2 的载波:

$$e_{2FSK}(t) = s_1(t) \cos \omega_1 t + s_2(t) \cos \omega_2 t$$

$$s_1(t) = \sum_n a_n g(t - nT_s)$$

$$s_2(t) = \sum_n \bar{a}_n g(t - nT_s)$$

$$a_n = \begin{cases} 1 & \text{概率为 } p \\ 0 & \text{概率为 } 1-p \end{cases} \quad \bar{a}_n = \begin{cases} 1 & \text{概率为 } 1-p \\ 0 & \text{概率为 } p \end{cases} \quad g(t) = \begin{cases} 1 & 0 \leq t \leq T_s \\ 0 & \text{其他} \end{cases}$$

运行结果:

```
julia> BER
0.1064
```

```
julia> BER_theory
0.10287033054190721
```

示例5: Syslab中AWGN中FSK信号的调制和解调

```
# 设置函数参数
rng = MersenneTwister(1234)
M = 2; # Modulation order
k = log2(M); # Bits per symbol
EbNo = 5; # Eb/No (dB)
Fs = 16; # Sample rate (Hz)
nsamp = 8; # Number of samples per symbol
freqsep = 10; # Frequency separation (Hz)
# 生成随机的符号数据
data = rand(rng, 0:M-1, 5000, 1);
# 频移键控调制
txsig = fskmod(data, M, freqsep, nsamp, Fs);
# 通过AWGN信道传递信号
rxSig = awgn(txsig, EbNo + 10 * log10(k) - 10 * log10(nsamp), "measured", 1234);
# 解调接收的信
dataOut = fskdemod(rxSig, M, freqsep, nsamp, Fs);
# 计算误比特率
num, BER = biterr(data, dataOut);
# 确定理论误码率,并与估计的误比特率进行比较
BER_theory = berawgn(EbNo, "fsk", M, "noncoherent");
(BER, BER_theory)
```

4 通信系统-频带信号的发送与接收

二. 数字调制与解调

多进制相移键控信号 (M-PSK) : 正弦载波的相位随数字基带信号而变化的数字调制。

2-PSK: 二进制相移键控信号, 发送为二进制符号 “1”时, 取0相位; 发送为二进制符号 “0”时, 取pi相位。

$$a_n = \begin{cases} 1 & \text{概率为 } p \\ 0 & \text{概率为 } 1-p \end{cases}$$

$$\varphi_n = \begin{cases} 1^\circ & \text{发送1} \\ 180^\circ & \text{发送0} \end{cases}$$

$$e_{2PSK}(t) = \cos(\omega_c t + \varphi_n)$$

示例6: Syslab中比较PSK和PAM信号上的相位噪声效应

```
rng = MersenneTwister(1234)
len = 10000;
M = 16;
# 生成随机的符号数据
msg = rand(rng, 0:(M-1), len, 1);
# 相移键控调制
txpsk = pskmod(msg, M);
# 脉冲幅度调制
txpam = pammod(msg, M);
# 生成随机相位噪声
phasenoise = randn(rng, len, 1) * 0.015;
```

```
# 相位噪声加入已调信号中
rxpsk = txpsk * exp(2im * pi * phasenoise);
rxpam = txpam * exp(2im * pi * phasenoise);
# 相移键控解调
recovpsk = pskdemod(rxpsk, M);
# 脉冲幅度解调
recovpam = pamdemod(rxpam, M);
# 计算误差符号数目
numerrs_psk = symerr(msg, recovpsk);
numerrs_pam = symerr(msg, recovpam);
```

运行结果:

```
julia> numerrs_psk
364

julia> numerrs_pam
0
```

注: M-ASK、M-FSK、M-PSK均是对载波振幅、频率和相位中的一种特性进行调制, 属于单一调制。QAM、APSK属于幅度-相位调制方式, 是对幅度和相位的复合调制。

4 通信系统-频带信号的发送与接收

二. 数字调制与解调

正交幅度调制(QAM)是振幅和相位联合键控的调制方式。基带码元波形经过串并转换分成I、Q两路，经过电平转换，与对应的载波相乘，然后再相加完成QAM的调制。

$$s_k(t) = A_k \cos(\omega_0 t + \theta_k)$$



$$s_k(t) = A_k \cos(\theta_k) \cos(\omega_0 t) - A_k \sin(\theta_k) \sin(\omega_0 t)$$

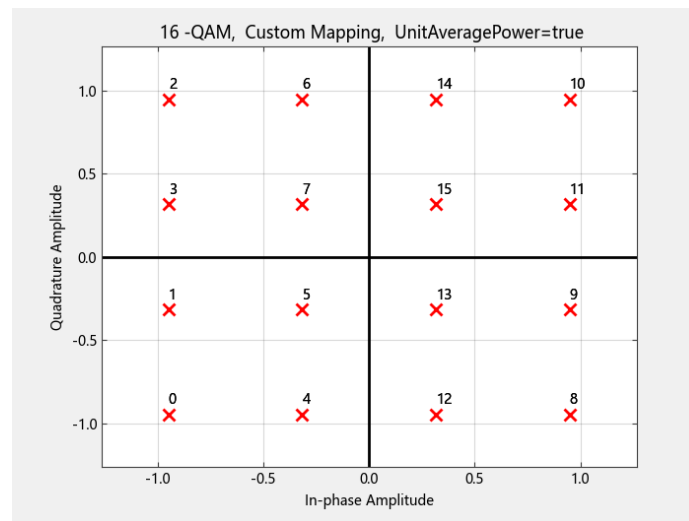
$$\text{令 } X_k = A_k \cos(\theta_k), Y_k = -A_k \sin(\theta_k)$$



$$s_k(t) = X_k \cos(\omega_0 t) - Y_k \sin(\omega_0 t)$$

示例7: Syslab中WLAN符号映射的QAM解调

```
# 生成一个随机符号的三维数组
rng = MersenneTwister(1234)
x = rand(rng, 0:15, 20, 4, 2);
# 根据 WLAN 标准为 16-QAM 星座图创建一个自定义的符号映射
wlanSymMap = [2 3 1 0 6 7 5 4 14 15 13 12 10 11 9 8];
# 对数据进行调制，并将星座图设置为具有单位平均信号功率，绘制出星座图
y = qammod(x, 16, wlanSymMap, UnitAveragePower=true, PlotConstellation=true);
# 对收到的信号进行解调
z = qamdemod(y, 16, wlanSymMap, UnitAveragePower=true);
# 验证解调后的信号是否与原始数据相等
isequal(x, z)
```



```
julia> isequal(x, z)
true
```

通常QAM 星座是矩形的，而 APSK 星座是圆形的。

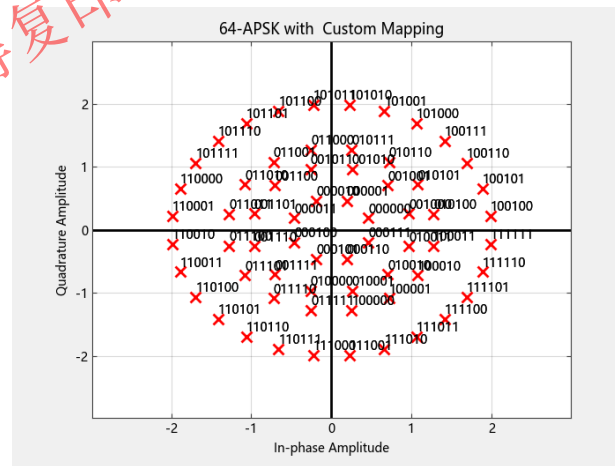
4 通信系统-频带信号的发送与接收

二. 数字调制与解调

幅度和相移键控(APSK) 通过调制载波的幅度和相位来传输数据。它结合了幅移键控(ASK) 和相移键控 (PSK), 与ASK 或 PSK 相比, 在给定调制阶数和信噪比的情况下降低误码率, 但代价是增加了复杂性。

示例8: Syslab中解调 64-APSK 自定义符号映射信号

```
# 定义调制阶数和 PSK 环半径的向量
rng = MersenneTwister(1234)
M = [8; 12; 16; 28]
modOrder = sum(M)
radii = [0.5; 1; 1.3; 2]
# 生成 100 个随机比特输入符号
x1 = randi(rng, [0 1], Int64(100 * log2(modOrder)), 1)
# 创建一个二进制映射的自定义符号映射向量
cmap = collect(range(0, stop=63))
# 加入高斯白噪声
snr1 = 20
rxSig = awgn(txSig, snr1, "measured")
# 调制数据, 绘制星座图
y = apskmod(x1, M, radii, SymbolMapping=cmap, InputType="bit", PlotConstellation=true)
# 对接收到的信号进行解调
z = apskdemod(y, M, radii, SymbolMapping=cmap, OutputType="bit")
# 与输入数据进行比较
isequal(x1, z)
```



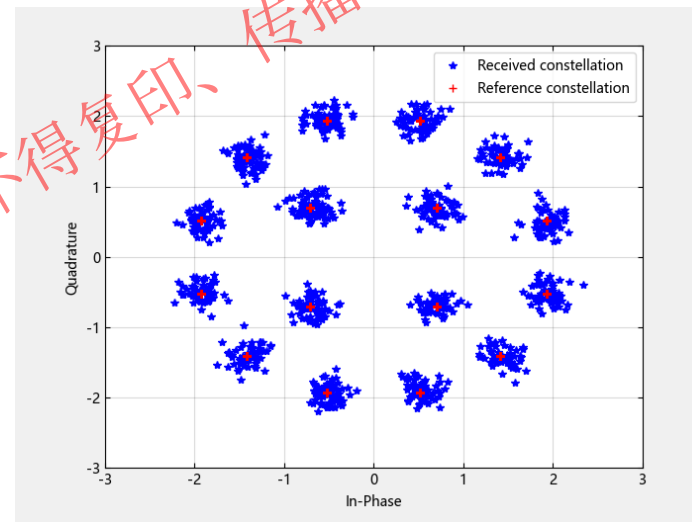
```
julia> isequal(x1, z)
true
```

4 通信系统-频带信号的发送与接收

二. 数字调制与解调

示例9: 解调 16-APSK 比特流信号

```
# 定义调制阶数和 PSK 环半径的向量。生成随机的 16 位数据符号
rng = MersenneTwister(1234)
M = [4; 12]
radii = [1; 2]
modOrder = sum(M)
x = randi(rng, [0 modOrder - 1], 1000, 1)
# 对数据进行 APSK 调制
txSig = apskmod(x, M, radii)
# 让调制信号通过噪声信道
snr1 = 20
rxSig = awgn(txSig, snr1, "measured")
# 绘制发射(参考)信号点和有噪声的接收信号点
plot(real(rxSig), imag(rxSig), "b*", markerfacecolor="auto")
hold("on")
grid()
plot(real(txSig), imag(txSig), "r+", markerfacecolor="auto")
xlim([-3 3])
ylim([-3 3])
xlabel("In-Phase")
ylabel("Quadrature")
legend(["Received constellation", "Reference constellation"], loc="upper right")
hold("off")
# 对接收到的信号进行解调，并与输入数据进行比较
z = apskdemod(rxSig, M, radii)
isequal(x, z)
```



用法:

```
z = apskdemod(y, M, radii)
z = apskdemod(y, M, radii, phaseoffset)
z = apskdemod(__; Name = Value)
```

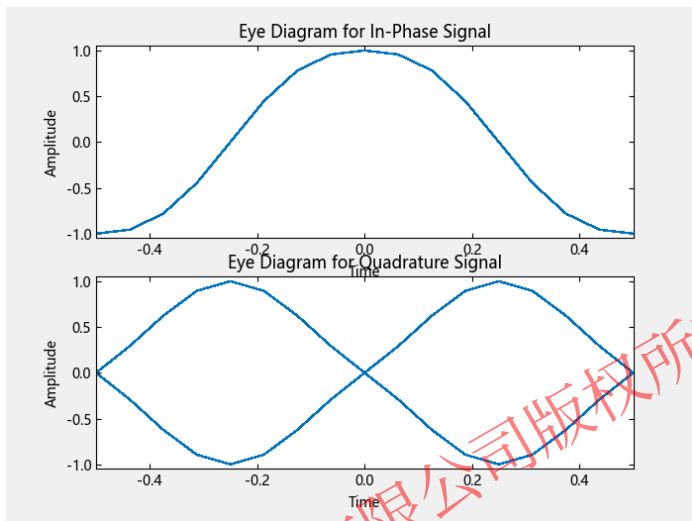
Syslab中函数的Name-Value是以关键字参数的形式使用，出现在其他参数之后，Name无需双引号，以等号与Value连接。

4 通信系统-频带信号的发送与接收

三. 结构体对象及相关函数的使用

以comm_CPMDemodulator为例，使用偏移正交移相键控来调制一个信号：

- 1、创建 comm_CPMDemodulator 对象并设置其属性；
- 2、用 step 调用该对象



运行结果：

```
julia> isequal(x[1:(end - delay)], z[(delay + 1):end])
true
```

注：在通信库中，以“comm_”开头的均为结构体对象，其使用方式均与comm_OQPSKModulator相同。结构体对象中 Name-Value 对是以等号连接。

示例10：应用 GFSK 调制和解调

```
# 创建 GFSK 调制器和解调器对象
rng = MT19937ar(1234)
gfskMod = comm_CPMMModulator(
    ModulationOrder=2,
    FrequencyPulse="Gaussian",
    BandwidthTimeProduct=0.5,
    ModulationIndex=1,
    BitInput=true
);
gfskDemod = comm_CPMDemodulator(
    ModulationOrder=2,
    FrequencyPulse="Gaussian",
    BandwidthTimeProduct=0.5,
    ModulationIndex=1,
    BitOutput=true
);
# 生成随机位数据并应用 GFSK 调制，使用眼图查看星座
numSym = 100;
x = randi(rng, [0 1], numSym * gfskMod.SamplesPerSymbol, 1);
y = step(gfskMod, x);
eyediagram(y, 16)
# 解调 GFSK 调制的信号
z = step(gfskDemod, y);
delay = 16;
isequal(x[1:(end-delay)], z[(delay+1):end])
```


4 通信系统-频带信号的发送与接收

模拟通带调制相关主要Syslab函数，其余函数请参阅Syslab-通信工具箱帮助文档

函数及调用方式	说明
ammod	调幅
amdemod	幅度解调
fmmod	调频
fmdemod	频率解调
pmmmod	调相
pmdemod	相位解调
ssbmod	单边带调制
ssbdemod	单边带解调

4 通信系统-频带信号的发送与接收

数字基带调制相关主要Syslab函数，其余函数请参阅Syslab-通信工具箱帮助文档

函数及调用方式	说明
apskmod	幅度相移键控 (APSK) 调制
apskdemod	幅度相移键控 (APSK) 解调
qammod	正交幅度调制
qamdemod	正交幅度解调
mskmod	最小移位键控调制
mskdemod	最小移位键控解调
dpskmod	差分相移键控调制
dpskdemod	差分相移键控解调

4 通信系统-频带信号的发送与接收

常见结构体对象，其余结构体对象请参阅Syslab-通信工具箱帮助文档

函数及调用方式	说明
comm_PSKModulator	M进制相移键控调制
comm_PSKDemodulator	M进制相移键控解调
comm_FModulator	用 FM 法调制信号
comm_FMDemodulator	解调基带 FM 信号
comm_OFDMModulator	使用 OFDM 法调制信号
comm_OFDMDemodulator	使用 OFDM 方法解调
comm_GeneralQAMModulator	使用任意 QAM 星座进行调制
comm_GeneralQAMDemodulator	使用任意 QAM 星座进行解调



目录

1. 通信工具箱功能概况

2. 信源编码

3. 信道编码

4. 频带信号的发送与接收

5. 性能仿真分析

6. 通信系统仿真综合案例

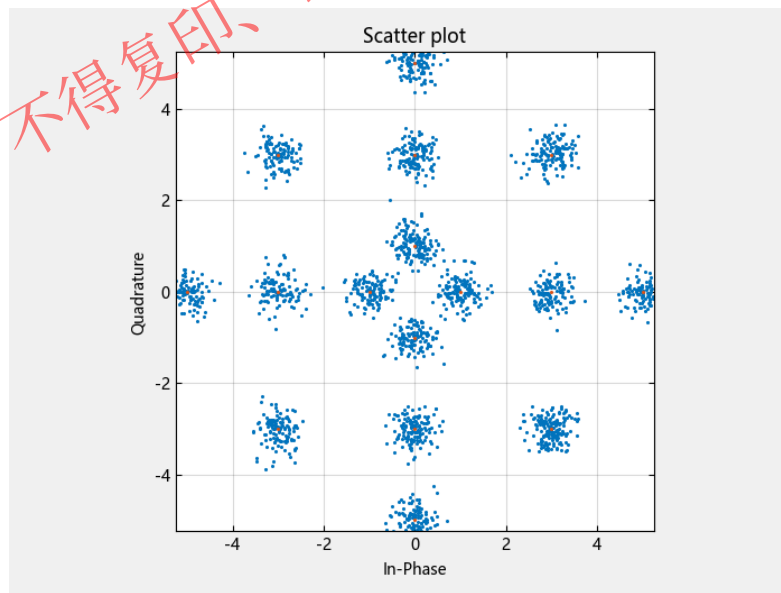
5 通信系统-性能仿真分析

一. 误码率

误码率又称码元差错率，是指在传输的码元总数中错误接收的码元数所占的比例。它是衡量数据通信系统在正常工作情况下的传输可靠性的指标。

示例1: Syslab中AWGN信道中通用QAM调制的码率估计

```
Random.seed!(1234)
c = [-5 -5im 5 5im -3 -3 - 3im -3im 3 - 3im 3 3 + 3im 3im -3 + 3im -1 -1im 1 1im];
sigpower = pow2db(mean(abs.(c).^2));
M = length(c);
# 生成随机符号
data = randi([0 M - 1], 2000, 1);
# 使用genqammod函数调制数据
modData = genqammod(data, c);
# 通过具有 20 dB SNR 的 AWGN 通道传递信号
rxSig = awgn(modData, 20, sigpower);
# 显示接收信号的星座图
scatterplot(rxSig);
hold("on")
# 参考星座的星座图
scatterplot(c)
grid("on")
hold("off")
# 使用genqamdmod函数解调接收到的信号
demodData = genqamdmod(rxSig, c);
# 确定符号错误的数量及误码率
numErrors1, ser1 = symerr(data, demodData)
# 通过具有 10 dB SNR 的 AWGN 通道传递信号
rxSig = awgn(modData, 10, sigpower);
## 使用genqamdmod函数解调接收到的信号
demodData = genqamdmod(rxSig, c);
# 确定符号错误的数量及误码率
numErrors2, ser2 = symerr(data, demodData)
```



运行结果:

```
julia> numErrors1, ser1
(5, 0.0025)

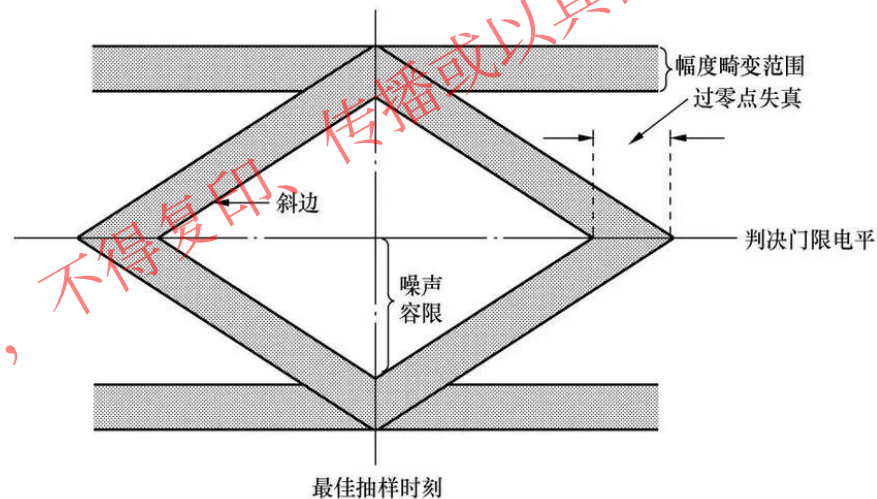
julia> numErrors2, ser2
(478, 0.239)
```

5 通信系统-性能仿真分析

二. 眼图

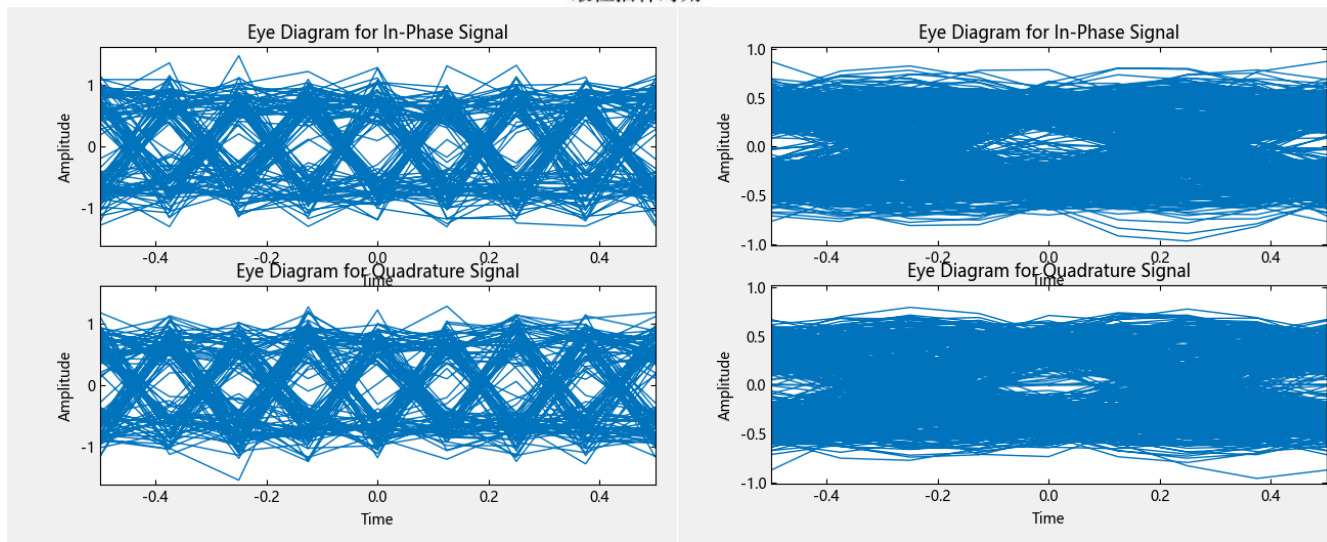
眼图充分反映通信链路上传输的所有数字信号的整体特征。眼图上能检视出数字传输码间串扰和信道噪声的影响，进而估算系统好坏程度。通常产生眼图的方式是示波器。

- 最佳抽样时刻应在“眼睛”张开最大的时刻。
- 眼图斜边的斜率决定定时误差的灵敏度。斜率越大，越灵敏。
- 抽样时刻上，眼图上下两分支阴影区的垂直高度，表示最大信号畸变。
- 眼图中央的横轴位置应对应判决门限电平。
- 在抽样时刻上，上下两分支离门限最近的一根线迹至门限的距离表示各相应电平的噪声容限。



示例2: Syslab中眼图的生成

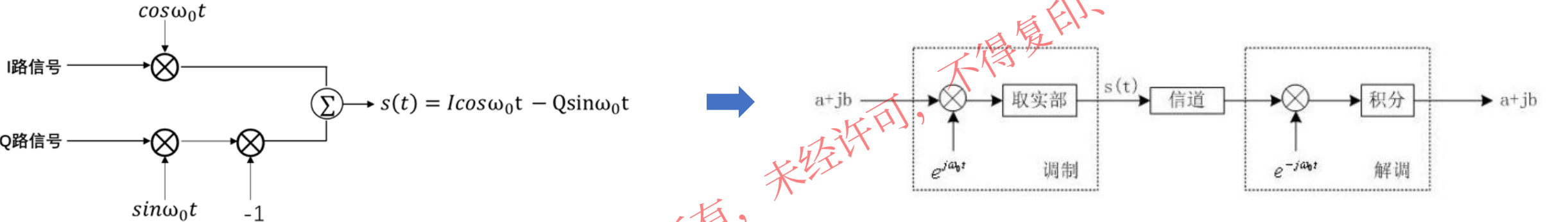
```
rng = MT19937ar(1234)
data = randi(rng, [0 3], 1000, 1);
modsig = pskmod(data, 4, pi / 4);
tmp = awgn(modsig, 10)
sps = 4;
txfilter = comm_RaisedCosineTransmitFilter( OutputSamplesPerSymbol=sps);
txsig = step(txfilter, tmp);
# 生成未过滤的OPSK信号眼图
eyediagram(tmp, 2 * sps)
# 生成过滤后的OPSK信号眼图
eyediagram(txsig, 2 * sps)
```



5 通信系统-性能仿真分析

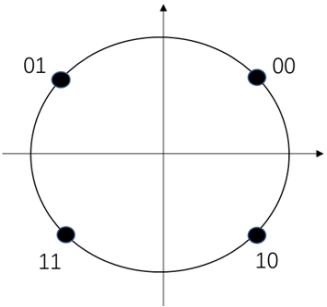
三.星座图

IQ调制就是数据分为两路，分别进行载波调制，两路载波相互正交。I是in-phase（同相），q是 quadrature（正交）。把IQ信号用一个复数 $I+jQ$ 来表示，那么这个 $I+jQ$ 信号在复平面上就对应一个点。



QPSK的IQ路信号与相位的关系

二进制序列	IQ信号	s(t)相位
00	$+\frac{1}{\sqrt{2}}, +\frac{1}{\sqrt{2}}$	$\frac{\pi}{4}$
01	$-\frac{1}{\sqrt{2}}, +\frac{1}{\sqrt{2}}$	$\frac{3\pi}{4}$
11	$-\frac{1}{\sqrt{2}}, -\frac{1}{\sqrt{2}}$	$\frac{5\pi}{4}$
10	$+\frac{1}{\sqrt{2}}, -\frac{1}{\sqrt{2}}$	$\frac{7\pi}{4}$



5 通信系统-性能仿真分析

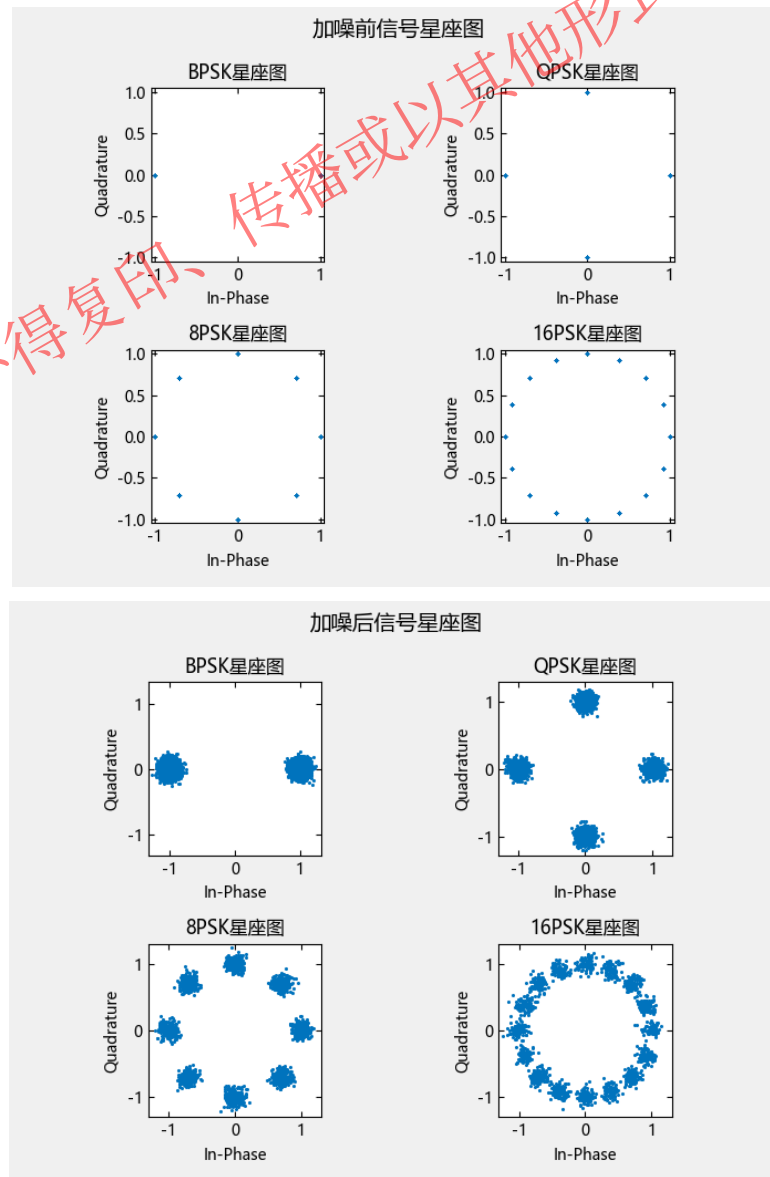
三.星座图

数字调制是在I、Q平面上将数据映射为许多离散的点，称这些点为**星座**。当信号从一个点移向另一个点时，**幅度调制**和**相位调制**就同时完成了。I、Q信号分别是在X轴和Y轴上的投影，合成矢量的幅度表示载波的幅度，合成矢量与X轴的夹角表示载波相位。

示例3: Syslab中PSK调制星座图的生成

```
rng = MT19937ar(1234)
data = randi(rng,[0 1], 6000, 1)
modsig1 = pskmod(data, 2; InputType="bit");
modsig2 = pskmod(data, 4; InputType="bit");
modsig3 = pskmod(data, 8; InputType="bit");
modsig4 = pskmod(data, 16; InputType="bit");
subplot(2,2,1)
scatterplot(modsig1)
title("BPSK星座图")
subplot(2,2,2)
scatterplot(modsig2)
title("QPSK星座图")
subplot(2,2,3)
scatterplot(modsig3)
title("8PSK星座图")
subplot(2,2,4)
scatterplot(modsig4)
title("16PSK星座图")
sgtitle("加噪前信号星座图"; fontsize=14)
tightlayout()
```

```
# 加噪后
txsig1 = awgn(modsig1, 20)
txsig2 = awgn(modsig2, 20)
txsig3 = awgn(modsig3, 20)
txsig4 = awgn(modsig4, 20)
figure()
subplot(2,2,1)
scatterplot(txsig1)
title("BPSK星座图")
subplot(2,2,2)
scatterplot(txsig2)
title("QPSK星座图")
subplot(2,2,3)
scatterplot(txsig3)
title("8PSK星座图")
subplot(2,2,4)
scatterplot(txsig4)
title("16PSK星座图")
sgtitle("加噪后信号星座图"; fontsize=14)
tightlayout()
```



5 通信系统-性能仿真分析

三. 星座图

示例4: Syslab中绘制眼图和散点图

设置RRC滤波器、调制方案和绘图参数

span = 10;

rolloff = 0.2;

sps = 8;

M = 4;

k = log2(M);

phOffset = pi/4;

n = 1;

offset = 0;

#创建滤波器系数

filtCoeff = rcosdesign(rolloff,span,sps);

rng=MT19937ar(5489)

data = randi(rng,[0 M-1],5000,1);

#QPSK调制

dataMod = pskmod(data,M,phOffset);

过滤调制数据

txSig = upfirdn(vec(dataMod),filtCoeff,sps);

EbNo = 20;

snr1 = EbNo + 10*log10(k) - 10*log10(sps);

rxSig = awgn(txSig, snr1, "measured");

应用 RRC 接收滤波器

rxSigFilt = upfirdn(vec(rxSig), filtCoeff, 1, sps);

解调过滤后的信号

dataOut = pskdemod(rxSigFilt, M, phOffset, "gray");

使用该scatterplot函数显示滤波前后信号的散点图

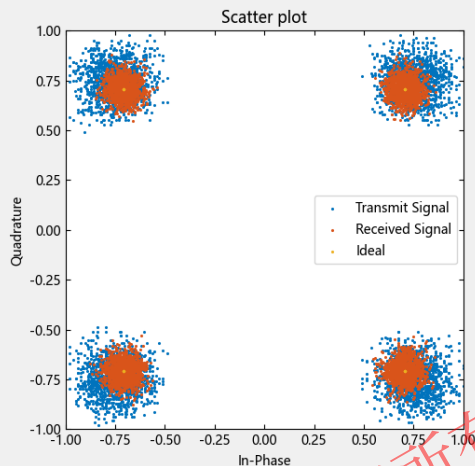
scatterplot(sqrt(sps)*txSig[sps*span+1:end-sps*span],sps,offset);

hold("on")

scatterplot(rxSigFilt[span+1:end-span], n, offset)

scatterplot(dataMod,n,offset)

legend(["Transmit Signal","Received Signal","Ideal","location","best"])



xlim([-1 1])

ylim([-1 1])

hold("off")

figure()

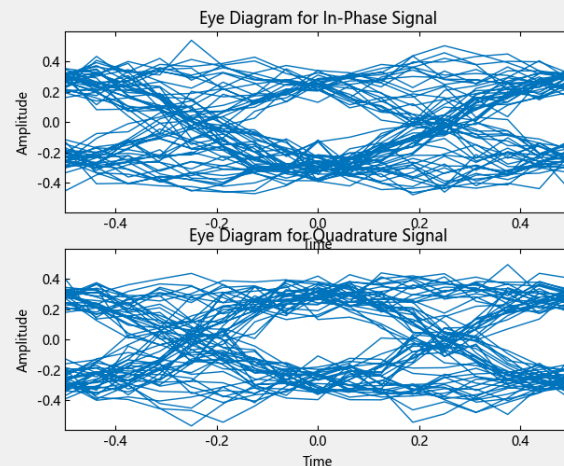
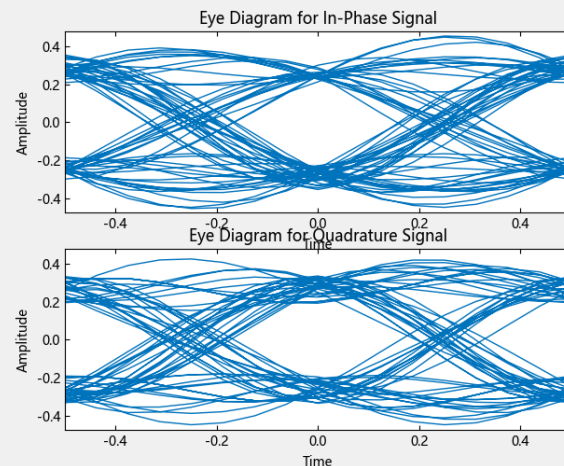
显示两个符号周期内传输信号眼图的 1000 个点

eyediagram(txSig[sps*span+1:sps*span+1000], 2 * sps)

figure()

显示接收信号眼图的 1000 点

eyediagram(rxSig[sps*span+1:sps*span+1000], 2 * sps)



目录

1. 通信工具箱功能概况

2. 信源编码

3. 信道编码

4. 频带信号的发送与接收

5. 性能仿真分析

6. 通信系统仿真综合案例

6 通信系统-通信系统仿真综合案例

一、通信系统仿真综合案例

```
#-----1、生成随机信号-----
rng = MT19937ar(1234)
y = randn(rng, 8000, 1)
#-----2、对信号进行信源编码-dpcm编码-----
initcodebook = collect(-1:0.005:2)
# 优化参数
predictor, codebook, partition = dpcmopt(vec(y), 1, initcodebook)
encodex, = dpcmenco(vec(y), codebook, partition, vec(predictor))
# 十进制转二进制
i_b = int2bit(encodex, 9)
#-----3、对信号进行差错控制编码-卷积编码-----
# 生成网格结构
trellis1 = poly2trellis([5 4], [23 35 0; 0 5 13])
# 卷积编码
codedout, = convenc(i_b, trellis1)
#-----4、对信号进行bpsk、qpsk、8psk、16psk调制-----
myBPSK = comm_BPSKModulator()
bpskData = step(myBPSK, codedout)
myQPSK = comm_QPSKModulator(; BitInput=true)
qpskData = step(myQPSK, codedout)
my8PSK = comm_PSKModulator(; ModulationOrder=8, BitInput=true)
pskData = step(my8PSK, codedout)
my16PSK = comm_PSKModulator(; ModulationOrder=16, BitInput=true)
psk16Data = step(my16PSK, codedout)
```

```
#-----5、调制信号通过信噪比为10db的AWGN-----
# 绘制psk、qpsk、8psk、16psk的散点图#通过AWGN后的星座图
rxsig1 = awgn(bpskData, 12)
figure()
sgtitle("通过AWGN后的星座图")
tightlayout()
subplot(2,2,1)
scatterplot(rxsig1)
grid("on")
title("BPSK")
rxsig2 = awgn(qpskData, 12)
subplot(2,2,2)
scatterplot(rxsig2)
grid("on")
title("QPSK")
rxsig3 = awgn(pskData, 12)
subplot(2,2,3)
scatterplot(rxsig3)
grid("on")
title("8PSK")
rxsig4 = awgn(psk16Data, 12)
subplot(2,2,4)
scatterplot(rxsig4)
grid("on")
title("16PSK")
tightlayout()
```

6 通信系统-通信系统仿真综合案例

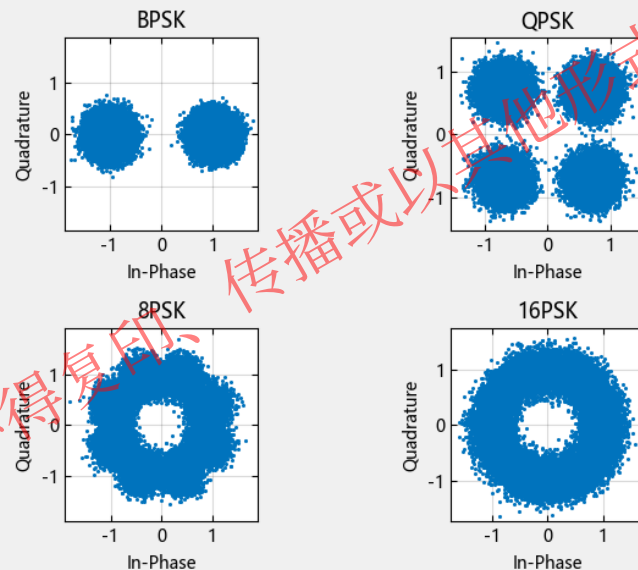
一、通信系统仿真综合案例

```
# 理想星座图
figure()
sgtitle("理想星座图")
tightlayout()
subplot(2,2,1)
scatterplot(bpskData)
grid("on")
title("BPSK")
subplot(2,2,2)
scatterplot(qpskData)
grid("on")
title("QPSK")
subplot(2,2,3)
scatterplot(pskData)
grid("on")
title("8PSK")
subplot(2,2,4)
scatterplot(psk16Data)
grid("on")
title("16PSK")
tightlayout()
```

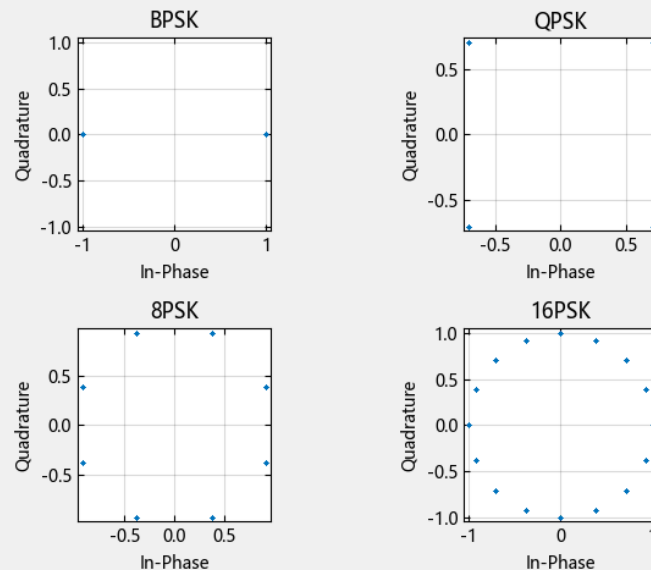
由通过理想星座图及噪声信道后的星座图可以看出，随调制阶数增大，相邻星座点之间的欧几里得距离减小，即噪声容限减小。



通过AWGN后的星座图



理想星座图



6 通信系统-通信系统仿真综合案例

一、通信系统仿真综合案例

```
#-----6、对信号进行bpsk、qpsk、8psk、16psk解调-----
myBPSKD1 = comm_BPSKDemodulator()
bpskdData1 = step(myBPSKD1, rxsig1)
myQPSKD1 = comm_QPSKDemodulator(; BitOutput=true)
qpskdData1 = step(myQPSKD1, rxsig2)
my8PSKD1 = comm_PSKDemodulator(; ModulationOrder=8, BitOutput=true)
pskdData1 = step(my8PSKD1, rxsig3)
my16PSKD1 = comm_PSKDemodulator(; ModulationOrder=16, BitOutput=true)
pskd16Data1 = step(my16PSKD1, rxsig4)
# -----7、对信号进行差错控制编码-维特比译码-----
tbdepth = 16
dataout1, = vitdec(bpskdData1, trellis1, tbdepth, "cont", "hard")
dataout2, = vitdec(qpskdData1, trellis1, tbdepth, "cont", "hard")
dataout3, = vitdec(pskdData1, trellis1, tbdepth, "cont", "hard")
dataout4, = vitdec(pskd16Data1, trellis1, tbdepth, "cont", "hard")
K = log2(trellis1.numInputSymbols)
decDelay = Int(K * tbdepth)
# 编码的误比特率
berCoded1 = biterr(i_b[1:(end - decDelay)], dataout1[(decDelay + 1):end])
berCoded2 = biterr(i_b[1:(end - decDelay)], dataout2[(decDelay + 1):end])
berCoded3 = biterr(i_b[1:(end - decDelay)], dataout3[(decDelay + 1):end])
berCoded4 = biterr(i_b[1:(end - decDelay)], dataout4[(decDelay + 1):end])
[berCoded1[1] berCoded2[1] berCoded3[1] berCoded4[1]]
```

运行结果:

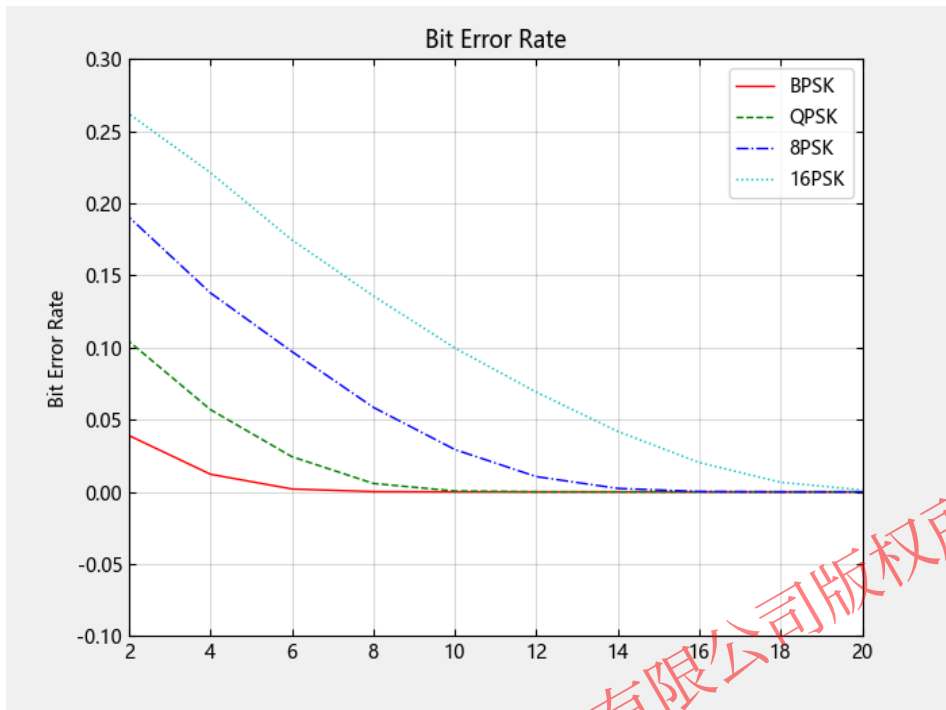
```
julia> [berCoded1[1] berCoded2[1] berCoded3[1] berCoded4[1]]
1x4 Matrix{Int64}:
 0  0  6 16507
```

在AWGN信噪比为15db时, bpsk以及qpsk的相邻星座点易于区分, 对解调干扰较小, 而8psk及16psk解调的误码率逐渐增大。

```
#-----绘制误比特率图,分析MPSK调制解调误码率与信噪比的关系-----
snrmax = 20
snrdet = 2
ber1 = zeros(Int(snrmax / snrdet) + 1)
ber2 = zeros(Int(snrmax / snrdet) + 1)
ber3 = zeros(Int(snrmax / snrdet) + 1)
ber4 = zeros(Int(snrmax / snrdet) + 1)
snrx = [0:snrdet:snrmax;]
figure()
for snr in 0:snrdet:snrmax
    # bpsk调制
    psk2 = comm_BPSKModulator()
    psk2_Data = step(psk2, i_b)
    # qpsk调制
    psk4 = comm_QPSKModulator(; BitInput=true)
    psk4_Data = step(psk4, i_b)
    # 8psk调制
    psk8 = comm_PSKModulator(; ModulationOrder=8, BitInput=true)
    psk8_Data = step(psk8, i_b)
    # 16psk调制
    psk16 = comm_PSKModulator(; ModulationOrder=16, BitInput=true)
    psk16_Data = step(psk16, i_b)
    #加噪,信噪比为snr
    rx1 = awgn(psk2_Data, snr)
    rx2 = awgn(psk4_Data, snr)
    rx3 = awgn(psk8_Data, snr)
    rx4 = awgn(psk16_Data, snr)
```

6 通信系统-通信系统仿真综合案例

一、通信系统仿真综合案例



随信噪比增加，bpsk、qpsk、8psk、16psk的误码率逐渐降低；
信噪比相同时(0-8db范围内)，psk的误码率随调制阶数M增大而增大。

```
# bpsk解调
pskd2 = comm_BPSKDemodulator()
pskd2_Data = step(pskd2, rx1)
# qpsk解调
pskd4 = comm_QPSKDemodulator('BitOutput=true')
pskd4_Data = step(pskd4, rx2)
# 8psk解调
pskd8 = comm_PSKDemodulator('ModulationOrder=8, BitOutput=true')
pskd8_Data = step(pskd8, rx3)
# 16psk解调
pskd16 = comm_PSKDemodulator('ModulationOrder=16, BitOutput=true')
pskd16_Data = step(pskd16, rx4)
#对比传输过程的位误差
(NumberError1, ErrorRate1, ErrorLoaction1) = biterr(i_b, pskd2_Data)
(NumberError2, ErrorRate2, ErrorLoaction2) = biterr(i_b, pskd4_Data)
(NumberError3, ErrorRate3, ErrorLoaction3) = biterr(i_b, pskd8_Data)
(NumberError4, ErrorRate4, ErrorLoaction4) = biterr(i_b, pskd16_Data)
ber1[Int(snr / snrdet) + 1] = ErrorRate1
ber2[Int(snr / snrdet) + 1] = ErrorRate2
ber3[Int(snr / snrdet) + 1] = ErrorRate3
ber4[Int(snr / snrdet) + 1] = ErrorRate4
end
plot(snrx, ber1, "r-")
hold("on")
grid("on")
plot(snrx, ber2, "g--")
plot(snrx, ber3, "b-")
plot(snrx, ber4, "c:")
legend(['BPSK', 'QPSK', '8PSK', '16PSK'])
axis([snrdet snrmax -0.1 0.3])
ylabel('Bit Error Rate')
title('Bit Error Rate')
```


建立知识规范，营造协同生态
积累工业模型，发展可控平台
融入中国创新，打造先进软件

Thanks !



@苏州同元软控信息技术有限公司版权所有，侵权必究